
[1] AUTHENTICATION & USER PROFILE API

CODE:

```
router.post("/signup", async (req, res) => {
  try {
    const { name, email, phone, password, role } = req.body;
    const hashedPassword = await bcrypt.hash(password, 10);
    const user = new User({ name, email, phone, password: hashedPassword, role });
    await user.save();
    res.status(201).json({ message: "User created" });
  } catch (e) { res.status(500).json(e); }
});

router.post("/login", async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user || !(await bcrypt.compare(req.body.password, user.password))) {
    return res.status(401).json({ message: "Invalid credentials" });
  }
  const token = jwt.sign({ userId: user._id, role: user.role }, process.env.JWT_SECRET);
  res.json({ token, user });
});

router.post("/generate-abha", authMiddleware, async (req, res) => {
  const abhald = `${Math.floor(10 + Math.random() * 90)}-${Math.floor(1000 + Math.random() * 9000)}-${Math.floor(1000 + Math.random() * 9000)}`;
  await User.findByIdAndUpdate(req.user.userId, { healthId: abhald });
  res.json({ healthId: abhald });
});
```

DESCRIPTION:

This module manages security. The 'Signup' API uses Bcrypt to 'salt and hash' passwords so they are never stored in plain text. The 'Login' API uses JSON Web Tokens (JWT) to create a secure session. The 'ABHA Generation' logic creates a unique 14-digit identifier compliant with NDHM standards, ensuring every patient has a digital medical identity.

[2] EMERGENCY SOS & REAL-TIME RESPONSE API

CODE:

```
router.post("/", async (req, res) => {
  try {
    const { patientName, phone, emergencyType, location } = req.body;
    const newEmergency = new Emergency({ patientName, phone, emergencyType, location });
    const saved = await newEmergency.save();

    const io = req.app.get("io");
    io.emit("new_emergency", saved);
    res.status(201).json(saved);
  } catch (e) { res.status(500).json(e); }
```

```
});
```

DESCRIPTION:

The SOS API is time-critical. It captures the patient's GeoJSON coordinates. The key logic here is the 'Socket.io Emission'. Instead of waiting for the hospital to refresh, the backend 'pushes' the alert to the Admin dashboard instantly. It creates a high-priority incident in the database to be tracked by ambulances.

[3] HOSPITAL GEOSPATIAL SEARCH API

CODE:

```
router.get("/", async (req, res) => {
  const { lat, lng, radius = 5000 } = req.query;
  let query = {};
  if (lat && lng) {
    query.location = {
      $near: {
        $geometry: { type: "Point", coordinates: [parseFloat(lng), parseFloat(lat)] },
        $maxDistance: parseInt(radius)
      }
    };
  }
  const hospitals = await Hospital.find(query);
  res.json(hospitals);
});
```

DESCRIPTION:

This API uses MongoDB's '\$near' operator. It performs a 2D-Sphere earth calculation to find hospitals within X meters of the user's current location. This allows patients to find help instantly during an emergency. The results are ordered by physical distance automatically by the database engine.

[4] APPOINTMENT SCHEDULING & LIFECYCLE API

CODE:

```
router.post("/", authMiddleware, async (req, res) => {
  const { doctorId, hospitalId, date, time, reason } = req.body;
  const appointment = new Appointment({
    patient: req.user.userId,
    doctor: doctorId,
    hospital: hospitalId,
    date, time, reason
  });
  await appointment.save();
  req.app.get("io").emit("appointment_created", appointment);
  res.status(201).json(appointment);
});

router.put("/:id/status", async (req, res) => {
  const appointment = await Appointment.findByIdAndUpdate(req.params.id, { status: req.body.status }, { new: true });
  req.app.get("io").emit("appointment_updated", appointment);
  res.json(appointment);
});
```

DESCRIPTION:

This manages the lifecycle of a medical consultation. Appointments transition from 'Pending' to 'Confirmed' to 'Completed'. The backend uses WebSockets to ensure patients get notified on their phones the instant a doctor approves their request. It handles role-based filtering (Patients see their history, Doctors see their schedule).

[5] MEDICAL RECORD & CLINICAL HISTORY API

CODE:

```
router.post("/", authMiddleware, async (req, res) => {
  const { appointmentId, patientId, diagnosis, medicines } = req.body;
  const record = new MedicalRecord({ patient: patientId, doctor: req.user.doctorId, appointment: appointmentId,
diagnosis, medicines });
  await record.save();

  if (appointmentId) {
    await Appointment.findByIdAndUpdate(appointmentId, { status: 'completed' });
  }
  res.status(201).json(record);
});
```

DESCRIPTION:

This is the most critical logic for data integrity. When a doctor creates a medical record, the backend logic 'Links' it to the source appointment. It then 'Auto-Closes' the appointment status. This ensures that a patient's medical history is always perfectly synced with their hospital visits.

[6] REAL-TIME CHAT & MESSAGING API

CODE:

```
io.on("connection", (socket) => {
  socket.on("join_room", (room) => { socket.join(room); });
  socket.on("send_message", async (data) => {
    const newMessage = new Message({ ...data });
    await newMessage.save();
    io.to(data.room).emit("receive_message", newMessage);
  });
});
```

DESCRIPTION:

Instead of standard REST, this uses WebSockets for a 'WhatsApp-like' experience. The backend creates Virtual Rooms for every patient-doctor pair. When a message is sent, it is both saved to MongoDB for history and broadcasted to the room for instant visibility.

[7] ADMIN DASHBOARD STATISTICS API

CODE:

```
router.get("/", async (req, res) => {
  const totalDoctors = await Doctor.countDocuments();
  const totalPatients = await User.countDocuments({ role: 'patient' });
  const pendingEmergencies = await Emergency.countDocuments({ status: 'incoming' });
  res.json({ totalDoctors, totalPatients, pendingEmergencies });
});
```

```
});
```

DESCRIPTION:

This API provides high-level insights. It uses the 'countDocuments' aggregation to pull real-time numbers. This allows hospital deans and administrators to monitor their entire medical facility's operational health at a single glance.

[8] WARD & BED MANAGEMENT API

CODE:

```
router.put("/:id", async (req, res) => {  
  const ward = await Ward.findById(req.params.id);  
  if (req.body.occupiedBeds) ward.occupiedBeds = req.body.occupiedBeds;  
  await ward.save();  
  res.json(ward);  
});
```

DESCRIPTION:

Manages the physical capacity of the hospital. It tracks bed counts across ICU, General, and Pediatric wards. When a patient is admitted, the 'occupiedBeds' count is updated, which reflected instantly on the 'Hospital Discovery' map for new patients.

END OF DOCUMENTATION
